```
==========================================================================
STEP 1: PREDICATE LOGIC REPRESENTATION
==========================================================================
Constants : mira, scroll_7, royal_archive
Variables : x (person), r (record)
Predicates: archivist/1, verifies/2, trusted/1, access/2, sealed/1,
            permitted/2, inspects/2, finds_mark/2, stored_in/2

  [1]  Every archivist who verifies a record is trusted
       forall x. forall r. ((archivist(x) & verifies(x,r)) => trusted(x))
  [2]  Every trusted person with access to a sealed record is permitted to inspect it
       forall x. forall r. (((trusted(x) & access(x,r)) & sealed(r)) => permitted(x,r))
  [3]  Anyone who inspects a record and finds a hidden mark in it verifies that record
       forall x. forall r. ((inspects(x,r) & finds_mark(x,r)) => verifies(x,r))
  [4]  Every record stored in the royal archive is sealed
       forall r. (stored_in(r,royal_archive) => sealed(r))
  [5]  scroll_7 is stored in the royal archive
       stored_in(scroll_7,royal_archive)
  [6]  Mira has access to scroll_7
       access(mira,scroll_7)
  [7]  Mira inspects scroll_7 and finds a hidden mark in it
       (inspects(mira,scroll_7) & finds_mark(mira,scroll_7))
  [8]  Mira is an archivist
       archivist(mira)

  GOAL: trusted(mira)   ("Mira is trusted")

==========================================================================
STEP 2: NEGATION OF THE GOAL (added to the knowledge base)
==========================================================================
  [9]  ~trusted(mira)
```

```
========================================================================
STEP 3: IMPLICATION REMOVAL   (A => B  ==>  ~A | B)
========================================================================
  [1]  forall x. forall r. (~(archivist(x) & verifies(x,r)) | trusted(x))
  [2]  forall x. forall r. (~((trusted(x) & access(x,r)) & sealed(r)) | permitted(x,r))
  [3]  forall x. forall r. (~(inspects(x,r) & finds_mark(x,r)) | verifies(x,r))
  [4]  forall r. (~stored_in(r,royal_archive) | sealed(r))
  [5]  stored_in(scroll_7,royal_archive)
  [6]  access(mira,scroll_7)
  [7]  (inspects(mira,scroll_7) & finds_mark(mira,scroll_7))
  [8]  archivist(mira)
  [9]  ~trusted(mira)

========================================================================
STEP 4: NEGATION MOVED INWARD
========================================================================
  [1]  forall x. forall r. ((~archivist(x) | ~verifies(x,r)) | trusted(x))
  [2]  forall x. forall r. (((~trusted(x) | ~access(x,r)) | ~sealed(r)) | permitted(x,r))
  [3]  forall x. forall r. ((~inspects(x,r) | ~finds_mark(x,r)) | verifies(x,r))
  [4]  forall r. (~stored_in(r,royal_archive) | sealed(r))
  [5]  stored_in(scroll_7,royal_archive)
  [6]  access(mira,scroll_7)
  [7]  (inspects(mira,scroll_7) & finds_mark(mira,scroll_7))
  [8]  archivist(mira)
  [9]  ~trusted(mira)
  (only negated atoms remain; nothing further to push in)

========================================================================
STEP 5: STANDARDIZE VARIABLES APART  (suffix = formula number)
========================================================================
  [1]  forall x1. forall r1. ((~archivist(x1) | ~verifies(x1,r1)) | trusted(x1))
  [2]  forall x2. forall r2. (((~trusted(x2) | ~access(x2,r2)) | ~sealed(r2)) | permitted(x2,r2))
  [3]  forall x3. forall r3. ((~inspects(x3,r3) | ~finds_mark(x3,r3)) | verifies(x3,r3))
  [4]  forall r4. (~stored_in(r4,royal_archive) | sealed(r4))
  [5]  stored_in(scroll_7,royal_archive)
  [6]  access(mira,scroll_7)
  [7]  (inspects(mira,scroll_7) & finds_mark(mira,scroll_7))
  [8]  archivist(mira)
  [9]  ~trusted(mira)
```

```
================================================================================
STEP 6: SKOLEMIZATION
================================================================================
  [1]  forall x1. forall r1. ((~archivist(x1) | ~verifies(x1,r1)) | trusted(x1))
  [2]  forall x2. forall r2. (((~trusted(x2) | ~access(x2,r2)) | ~sealed(r2)) | permitted(x2,r2))
  [3]  forall x3. forall r3. ((~inspects(x3,r3) | ~finds_mark(x3,r3)) | verifies(x3,r3))
  [4]  forall r4. (~stored_in(r4,royal_archive) | sealed(r4))
  [5]  stored_in(scroll_7,royal_archive)
  [6]  access(mira,scroll_7)
  [7]  (inspects(mira,scroll_7) & finds_mark(mira,scroll_7))
  [8]  archivist(mira)
  [9]  ~trusted(mira)
  (no existential quantifiers exist, so no Skolem terms are needed)

================================================================================
STEP 7: CNF CLAUSES  (universal quantifiers dropped, split on &)
================================================================================
  C1  (from formula 1):  ~archivist(x1) | ~verifies(x1,r1) | trusted(x1)
  C2  (from formula 2):  ~trusted(x2) | ~access(x2,r2) | ~sealed(r2) | permitted(x2,r2)
  C3  (from formula 3):  ~inspects(x3,r3) | ~finds_mark(x3,r3) | verifies(x3,r3)
  C4  (from formula 4):  ~stored_in(r4,royal_archive) | sealed(r4)
  C5  (from formula 5):  stored_in(scroll_7,royal_archive)
  C6  (from formula 6):  access(mira,scroll_7)
  C7  (from formula 7):  inspects(mira,scroll_7)
  C8  (from formula 7):  finds_mark(mira,scroll_7)
  C9  (from formula 8):  archivist(mira)
  C10  (from formula 9):  ~trusted(mira)
```

```
========================================================================
STEP 8: RESOLUTION REFUTATION
========================================================================
Strategy: set of support, breadth first. Support clause = C10 (negated goal).

Derivation (each line resolves two clauses on a complementary pair):

  R1:  resolve C10  with  C1   on  trusted(mira)
       C10 : ~trusted(mira)
       C1 : ~archivist(X) | ~verifies(X,Y) | trusted(X)
       ==> R1 : ~archivist(mira) | ~verifies(mira,X)

  R2:  resolve R1  with  C3   on  verifies(mira,X)
       R1 : ~archivist(mira) | ~verifies(mira,X)
       C3 : ~inspects(X,Y) | ~finds_mark(X,Y) | verifies(X,Y)
       ==> R2 : ~archivist(mira) | ~inspects(mira,X) | ~finds_mark(mira,X)

  R3:  resolve R2  with  C7   on  inspects(mira,scroll_7)
       R2 : ~archivist(mira) | ~inspects(mira,X) | ~finds_mark(mira,X)
       C7 : inspects(mira,scroll_7)
       ==> R3 : ~archivist(mira) | ~finds_mark(mira,scroll_7)

  R4:  resolve R3  with  C8   on  finds_mark(mira,scroll_7)
       R3 : ~archivist(mira) | ~finds_mark(mira,scroll_7)
       C8 : finds_mark(mira,scroll_7)
       ==> R4 : ~archivist(mira)

  R5:  resolve R4  with  C9   on  archivist(mira)
       R4 : ~archivist(mira)
       C9 : archivist(mira)
       ==> R5 : []  (empty clause)


  Clauses used in the proof : [1,3,7,8,9,10]
  Clauses NOT needed        : [2,4,5,6]  (rules 2, 4 and the facts feeding only them)

========================================================================
RESULT
========================================================================
  The empty clause [] was derived (step R5).
  KB + ~trusted(mira) is contradictory, so  trusted(mira)  is PROVED.
```

```
========================================================================
CROSS-CHECK WITH ORDINARY PROLOG (SLD resolution)
========================================================================
  ?- chk_trusted(mira).   true.
true
```